



[] KOM120C PEMROGRAMAN

K10 | Pekan Ke-11 Higher Order Function ++

Tim Pengajar Pemrograman IPB University

Learning Outcome

- Mahasiswa memahami konsep dan implementasi Higher Order Function (HOF).
- Mahasiswa mampu membuat program menggunakan HOF untuk problem yang lebih kompleks.



Higher Order Function (HOF) - REVIEW

HOF sering didefinisikan sebagai fungsi yang:

(a) memanggil fungsi lain sebagai parameter input

```
val gaji = Seq(20000, 70000, 40000)
val dobelGaji = (x: Int) => x * 2
val gajiBaru = gaji.map(dobelGaji)
println(gajiBaru)
```



```
List(40000, 140000, 80000)
```

(b) mengembalikan fungsi sebagai hasil (return)

```
def createMultiplier(factor: Int): Int => Int = {
  (x: Int) => x * factor
}
```

```
val multiplyByTwo = createMultiplier(2)
println(multiplyByTwo(4))
```



```
8
```

HOF :: foreach

Foreach digunakan untuk menerapkan (apply) sebuah fungsi ke masing-masing elemen pada sebuah koleksi data. **Apa bedanya dengan Map?**

```
val dt = List(2, 5, 3, 8, 2)

dt.foreach(print)
println()

dt.foreach(x => print(x*x))
println()

dt.foreach(x => print(s"$x "))
println()

dt.map(x => x*x).foreach(x => print(s"$x "))
```



```
25382
4259644
2 5 3 8 2
4 25 9 64 4
```

HOF :: sort?

Ada 3 HOF yang sering digunakan untuk pengurutan data (sorting):

- **sorted**: pengurutan dengan urutan default (misal: angka dari kecil ke besar)
- **sortBy**: pengurutan dengan menggunakan fungsi pembandingan untuk mengurutkan (misal: urutkan string berdasarkan panjangnya)
- **sortWith**: pengurutan dengan menggunakan kriteria perbandingan selain default (misal: dari besar ke kecil)

```
val dt = List(5, 3, 1, 4, 2)
println(dt.sorted)
println(dt.sortWith(_ > _))
```



```
List(1, 2, 3, 4, 5)
List(5, 4, 3, 2, 1)
```

```
val buah = List("pisang", "mengkudu", "jambu", "apel")
println(buah.map(_.length))
println(buah.sortBy(_.length))
```



```
List(6, 8, 5, 4)
List(apel, jambu, pisang, mengkudu)
```

HOF :: **groupBy**

groupBy digunakan untuk mengelompokkan elemen-elemen data dari sebuah koleksi dengan menggunakan sebuah kriteria tertentu. Misalnya mengelompokkan elemen berdasarkan apakah nilainya ganjil ataukah genap

```
val dt = List(1, 3, 2, 5, 2, 3, 4, 1, 6)
dt.groupBy(_%2).foreach(println)
println()
dt.groupBy(_%2).foreach{
  case (k,v) => println(k + " -> " + v)
}
```

```
(0,List(2, 2, 4, 6))
(1,List(1, 3, 5, 3, 1))
```

```
0 -> List(2, 2, 4, 6)
1 -> List(1, 3, 5, 3, 1)
```

```
// mengelompokkan berdasarkan panjang kata
val buah = List("pisang", "nangka", "jambu", "apel", "nanas")
buah.groupBy(_.length).foreach{ case (k,v) => println(s"$k -> $v") }
```

```
5 -> List(jambu, nanas)
6 -> List(pisang, nangka)
4 -> List(apel)
```

HashMap

Hasil dari `groupBy` adalah struktur `HashMap` (key -> value), salah satu implementasi dari struktur `Map` selain `TreeMap`, `ListMap`, etc. Akan dijelaskan pada pekan berikutnya.

```
val dt = List(1, 5, 10, 2, 5, 5, 1, 2, 3)
println(dt.groupBy(_%2))
```



```
HashMap(0 -> List(10, 2, 2), 1 -> List(1, 5, 5, 5, 1, 3))
```

```
// Mengolah value dari Map
val nilaiMutu = Map("A" -> 4, "B" -> 3, "C" -> 2, "D" -> 1, "E" -> 0)
val res = nilaiMutu.view.mapValues(_ + 1).toMap
println(res)
```



```
HashMap(E -> 1, A -> 5, B -> 4, C -> 3, D -> 2)
```

HashMap

```
// Mengurutkan Map by Key  
val nilaiMutu = Map("E" -> 0, "B" -> 3, "C" -> 2, "D" -> 1, "A" -> 4)  
val res = nilaiMutu.toSeq.sortWith(_. _1 < _. _1)  
println(res)
```



```
List((A,4), (B,3), (C,2), (D,1), (E,0))
```

```
// Menggabungkan 2 List menjadi Map  
val L1 = List("Bogor", "Jakarta", "Surabaya", "Bandung")  
val L2 = List(1, 4, 2, 3)  
val map = (L1 zip L2).toMap  
println(map)
```



```
Map("Bogor" -> 1, "Jakarta" -> 4, "Surabaya" -> 2, "Bandung" -> 3)
```


HOF :: untuk tipe data **String**

Beberapa di antaranya:

- **Length**: panjang string
- **Split**: memecah string berdasarkan karakter tertentu (tokenisasi)
- **Reverse**: membalik string
- **Contains**: apakah string mengandung substring tertentu atau tidak
- **startsWith/endsWith**: memeriksa apakah string dimulai/diakhir dengan substring tertentu
- **toLowerCase/toUpperCase**: mengubah case (kecil/besar)
- **indexOf**: mencari indeks kemunculan pertama karakter tertentu di dalam string
- ... (silakan explore sendiri)

Contoh #1

Diberikan sekumpulan string. Keluarkan string-string yang tidak mengandung substring tertentu secara terbalik dan diubah ke huruf besar, diurutkan mulai dari yang paling panjang ke yang paling pendek.

Ide
Solusi

Diberikan list dari string L dan sebuah substring str → filter L yang tidak mengandung str (**contains**) → urutkan berdasarkan panjang kata secara descending (**sortWith**) → balik huruf dalam kata (**reverse**) → ubah setiap elemen ke huruf besar (**toUpperCase**).

Scala

```
def balik(L: List[String], str: String): List[String] = {  
  L.filter(!_.contains(str)).sortWith(_.length>_.length)  
    .map(_.reverse).map(_.toUpperCase)  
}  
  
def main(args: Array[String]): Unit = {  
  val buah = List("pisang", "nangka", "jambu", "apel", "nanas")  
  println(balik(buah,"an"))  
}
```



List(UBMAJ, LEPA)

Contoh #2

Diberikan sekumpulan string. Keluarkan semua kelompok string yang merupakan anagram satu dengan yang lain (misal: "satu" $\Rightarrow$ "tuas") dalam lowerCase.

Contoh Input:

List("listen", "silent", "hello", "dear", "world", "study", "taste", "read", "dusty", "dare")

Contoh Output:

dear, read, dare
listen, silent
study, dusty

Ide
Solusi

Diberikan list dari string L $\rightarrow$ kelompokkan berdasarkan susunan huruf (**groupBy**) $\rightarrow$ filter dari **hashMap** yang nilainya lebih dari 1 kata (**values, filter**) $\rightarrow$ cetak dengan pemisah koma (**mkString**).

```
def anagram(L: List[String]): Unit = {  
  L.groupBy(_.toLowerCase.sorted).values.filter(_._2.length>1)  
    .foreach(grup => println(grup.mkString(", ")))  
}  
  
def main(args: Array[String]): Unit = {  
  val kata = List("listen", "silent", "hello", "dear", "world", "study", "taste",  
                  "read", "dusty", "dare")  
  anagram(kata)  
}
```

scala

Diskusi Kelas #1

Buat program Scala yang menerima input berupa deret sembarang bilangan bulat, dan menghitung frekuensi kemunculan setiap bilangan. Output diurutkan menurut bilangannya.

Contoh Input:

9
1 5 8 2 5 5 1 2 3

Contoh Output:

1 -> 2
2 -> 2
3 -> 1
5 -> 3
8 -> 1

Diskusi Kelas #2

Buat program Scala yang menerima input berupa deret sembarang bilangan bulat, dan menghitung banyaknya faktor dari setiap bilangan yang ada. Faktor dari N adalah banyaknya bilangan 1 sampai dengan N yang habis membagi N. Output diurutkan berdasarkan bilangan input.

Contoh Input:

4

5 8 2 3

Contoh Output:

2 -> 2

3 -> 2

5 -> 2

8 -> 4